

PROGETTO PER IL CORSO DI ARCHITETTURE PARALLELE - MINIMIZZAZIONE DI FIX-SET PER 2-SAT

LEONARDO DANELUTTI

danelutti.leonardo@spes.uniud.it

ABSTRACT. Il progetto affronta la risoluzione di istanze 2-SAT con l'obiettivo di identificare un insieme minimo di variabili (fix-set) che, una volta assegnate, rendono la formula soddisfacibile con una sola soluzione. L'obiettivo è quello di implementare un euristica per approssimare il risultato su istanze di grandi dimensioni, sfruttando la potenza di calcolo parallelo delle GPU.

Contents

1. Descrizione del problema	2
1.1. Richiami teorici	2
2. Algoritmo Overview	3
3. Euristiche	5
4. Esempio	6
5. Implementazione	7
5.1. Rappresentazione dei dati	7
5.2. Calcolo delle SCC	8
5.3. Calcolo dell'ordinamento topologico	10
5.4. Calcolo del backbone	10
5.5. Calcolo delle WCC	11
5.6. Calcolo delle euristiche	12
6. Risultati sperimentali	12
6.1. Metodo di generazione delle istanze	13
6.2. Grado di approssimazione delle euristiche	14
6.3. Analisi delle performance	15
7. Possibili miglioramenti	18
8. Valutazioni e osservazioni	18
9. Guida all'uso	19
9.1. Riproducibilità	20
References	20

1. DESCRIZIONE DEL PROBLEMA

Data un'istanza di formule booleane in forma normale congiuntiva con clausole di al massimo due letterali, si cerca un insieme di variabili, e un relativo assegnamento di verità, tale che la formula ammetta un'unica soluzione soddisfacibile.

Definizione 1.1. *Data una formula booleana P in forma normale congiuntiva (CNF) con clausole di al massimo due letterali (2-CNF), si cerca un sottoinsieme delle variabili in P , che chiameremo **fix-set** X , e un relativo assegnamento di verità f tale che la formula $P[x \mapsto f(x)]_{x \in X}$, ovvero la formula ottenuta da P sostituendo ogni variabile x in X con il suo valore assegnato da f , sia soddisfacibile e ammetta un'unica soluzione.*

Ad esempio, un fix-set banale per una formula soddisfacibile è l'insieme di tutte le variabili, il problema più interessante è trovare un fix-set di dimensione minima.

1.1. Richiami teorici.

Un'istanza 2-SAT può essere rappresentata tramite un **grafo delle implicazioni** $G = (V, E)$, dove i nodi V rappresentano i letterali (variabili e loro negazioni) e gli archi E rappresentano le implicazioni logiche derivanti dalle clausole. Una clausola $(a \vee b)$ genera gli archi $(\neg a \Rightarrow b)$ e $(\neg b \Rightarrow a)$.

E' facile vedere che un'istanza è soddisfacibile se e solo se, per ogni variabile x , i letterali x e $\neg x$ non possono raggiungersi reciprocamente nel grafo delle implicazioni. In altre parole, x e $\neg x$ devono appartenere a componenti fortemente connesse (SCC) distinte. Se x e $\neg x$ sono nella stessa SCC, allora la formula è insoddisfacibile.

Inoltre, presa un'istanza soddisfacibile, ogni letterale che si trova in una stessa SCC deve essere assegnato lo stesso valore di verità in ogni soluzione soddisfacibile. Si può quindi costruire un DAG (Directed Acyclic Graph) delle SCC, dove ogni nodo rappresenta una SCC e gli archi rappresentano le implicazioni tra di esse. Possiamo quindi ragionare su questo grafo condensato, per ogni nodo poi sarà necessario prendere un unico letterale rappresentante della SCC.

Alcune variabili, che chiameremo **backbone**, devono necessariamente assumere un certo valore in ogni soluzione soddisfacibile. Queste variabili possono essere identificate analizzando il grafo condensato.

Teorema 1.1.2. *Sia P un'istanza soddisfacibile di 2-SAT, e sia G il suo grafo delle implicazioni. Esiste un nodo v in G tale che v raggiunge $\neg v$ se e solo se v è falsa in ogni soluzione soddisfacibile, e quindi fa parte del backbone. Analogamente, $\neg v$ raggiunge v se e solo se v è vera in ogni soluzione soddisfacibile, e quindi fa parte del backbone.*

Dimostrazione.

$\Rightarrow$) Supponiamo che v raggiunga $\neg v$, cioè che esiste una catena di implicazioni $v \rightarrow l_1 \rightarrow \dots \rightarrow l_n \rightarrow \neg v$. Se assegniamo vero a v allora per le implicazioni $l_1, \dots, l_n$

devono essere assegnati a vero, e quindi $\neg v$ deve essere assegnato a vero, ma questo è una contraddizione.

$\Leftarrow$) Supponiamo che v sia falsa in ogni soluzione soddisfacibile, e supponiamo per assurdo che v non raggiunga $\neg v$.

- Se $\neg v$ raggiunge v , allora $\neg v$ è falsa in ogni soluzione soddisfacibile, e quindi v è vera in ogni soluzione soddisfacibile, ma questo è una contraddizione.
- Se $\neg v$ non raggiunge v , allora nessun altro letterale l raggiungibile da v può raggiungere il suo complementare, se così fosse avremmo una catena di implicazioni $v \xrightarrow{*} l \xrightarrow{*} \neg l \xrightarrow{*} \neg v$ (dove $\xrightarrow{*}$ indica la raggiungibilità). Quindi possiamo assegnare vero a v e questo non porta a contraddizioni, ma questo è una contraddizione con l'ipotesi che v è falsa in ogni soluzione soddisfacibile.

Lo stesso vale per il caso duale in cui $\neg v$ raggiunge v . $\square$

Tutti le variabili del backbone hanno lo stesso valore in ogni soluzione soddisfacibile, quindi non serve includerle nel fix-set, è sufficiente assegnarle e rimuoverle dal problema semplificandolo.

Un'altra affermazione importante è che se un node viene assegnato il valore di verità vero, allora tutti i nodi che raggiunge devono essere assegnati a vero, e in modo duale se un nodo viene assegnato a falso, allora tutti i nodi che lo raggiungono devono essere assegnati a falso. In generale quindi è preferibile aggiungere al fix-set nodi sorgente o pozzo nel DAG condensato, in modo che l'aggiunta del nodo porti a propagare (e quindi eliminare) i valori di verità a più nodi possibili.

2. ALGORITMO OVERVIEW

La soluzione implementata segue una pipeline di elaborazione del grafo delle implicazioni, che si articola in più fasi:

1. **Calcolo delle SCC:** Identificazione delle componenti fortemente connesse.
2. **Ordinamento Topologico:** Calcolo dei livelli topologici del DAG delle SCC per facilitare l'analisi della raggiungibilità.
3. **Calcolo del Backbone:** Identificazione del backbone, per rimuovere le variabili che assumono lo stesso valore in tutte le soluzioni.

Il calcolo del backbone, come anticipato, consiste di verificare se un letterale può raggiungere il suo complementare nel DAG condensato. Dato che dobbiamo calcolare la raggiungibilità tra coppie di nodi in un DAG, è possibile farlo partendo dai nodi sorgente e propagando la raggiungibilità fino a raggiungere i nodi pozzo. Di seguito lo pseudocodice dell'algoritmo per il calcolo del backbone.

Algoritmo 1: Backbone

Input: Un DAG $G = (V, E)$ con $n = |V|$ e l'ordinamento topologico di G

Output: Una lista di variabili che fanno parte del backbone

```
1  Inizializza un array reachability di dimensione  $n \times n$ , con false
2  Inizializza una lista vuota backbone
3  for each nodo  $u$  in  $G$  in ordine topologico do
4      reachability[u][u] = true
5      if reachability[u][complementare(u)] then
6           $u$  è sempre vero, complementare( $u$ ) è sempre falso
7          Aggiungi  $u$  e complementare( $u$ ) alla lista backbone
8      for each  $(u, v) \in E$  do
9          reachability[v] = reachability[v] OR reachability[u]
10     end for
11 end for
```

1. **Calcolo delle WCC:** Suddivisione del grafo rimanente in componenti debolmente connesse (Weakly Connected Components o WCC), che possono essere risolte indipendentemente.
2. **Ricerca Euristica:** L'algoritmo procede iterativamente finché tutti i nodi non sono stati assegnati. Ad ogni iterazione, viene selezionata una variabile da aggiungere al fix-set, e il grafo viene semplificato propagando le assegnazioni di verità. Il processo continua fino alla completa semplificazione del grafo. È importante notare che, essendo le variabili del backbone già state rimosse, questo processo non presenta contraddizioni logiche: non possono verificarsi situazioni in cui l'assegnazione di un letterale entri in conflitto con l'assegnazione del suo complementare.

Possiamo operare su ogni WCC in modo indipendente, ci posso essere due casi:

1. Un letterale x e il suo complementare $\neg x$ si trovano nella stessa WCC, in questo caso ciò vale per ogni letterale nella WCC. Prendiamo un letterale y nella stessa WCC di x , si ha quindi che $x \xrightarrow{*} y$ oppure $y \xrightarrow{*} x$ ma deve quindi valere anche $\neg x \xrightarrow{*} \neg y$ oppure $\neg y \xrightarrow{*} \neg x$, in entrambi i casi y e $\neg y$ si trovano nella stessa WCC.
2. Un letterale x e il suo complementare $\neg x$ si trovano in WCC distinte, in questo caso ogni letterale nella WCC di x non raggiunge il suo complementare per un motivo simile a quello visto sopra.

Per ogni iterazione se la WCC è del caso 1. possiamo scegliere un qualsiasi letterale nella WCC da aggiungere al fix-set, se invece è del caso 2. possiamo scegliere solo un letterale tra le due coppie di WCC. Ci riferiremo a «WCC complementare» per

indicare lo stesso WCC nel caso 1, e per indicare la WCC che contiene i complementari dei letterali nel caso 2.

Algoritmo 2: Backbone

```
1 while esistono nodi non assegnati do
2   for each coppia di WCC complementari  $C$  do
3     Applica l'euristica per selezionare un nodo candidato  $u$  da  $C$ 
4     Aggiungi il nodo selezionato al fix-set  $X$ 
5     Propaga le assegnazioni di verità a tutti i nodi raggiunti da  $u$ , e a tutti i nodi
      che raggiungono il complementare di  $u$ 
6     Rimuovi i nodi assegnati e aggiorna le WCC
7   end for
8 end while
```

3. EURISTICHE

Sono state implementate diverse euristiche per guidare la selezione delle variabili da aggiungere al fix-set, queste ad ogni iterazione vengono applicate come specificato sopra.

1. **Heuristic 1:** Seleziona un nodo sorgente. Ad ogni iterazione, lo stato di nodo sorgente viene ricalcolato analizzando gli archi uscenti verso nodi non assegnati
2. **Heuristic 2:** Seleziona il nodo con il più alto out-degree. Ad ogni iterazione, il conteggio degli archi uscenti viene dinamicamente aggiornato escludendo i vicini già assegnati
3. **Heuristic 3:** Seleziona il nodo con la più alta raggiungibilità, ovvero il nodo che raggiunge il maggior numero di vertici nel DAG condensato. In questa variante, il calcolo della raggiungibilità viene effettuato una sola volta all'avvio, e iterativamente viene applicata una maschera per escludere i nodi rimossi
4. **Heuristic 4:** Variante della Heuristic 3 che prevede il ricalcolo totale e dinamico della raggiungibilità dopo ogni round di assegnamento, offrendo maggiore precisione a fronte di un costo computazionale più elevato

E' importante notare come il complementare di nodo sorgente in un WCC è un nodo pozzo nel WCC complementare, e questo vale in modo analogo con tutte le metriche che utilizzano le euristiche, ad esempio se un nodo ha out-degree d , allora il suo complementare ha in-degree d , se un nodo raggiunge n altri nodi, allora il suo complementare è raggiunto da n nodi. Quindi in realtà, in ogni euristica, non

controlliamo solo «in avanti» ma anche «all'indietro», e questo è importante per guidare la selezione delle variabili in modo più efficace.

4. ESEMPIO

Nella Figura 1 viene mostrato un esempio di un possibile grafo delle implicazioni, nodi bianchi rappresentano i letterali non negati mentre nodi rossi rappresentano i letterali negati, gli archi rappresentano le implicazioni logiche. Il primo passo è quello di calcolare le componenti fortemente connesse, in questo caso abbiamo 4 SCC, rappresentate dalle zone colorate in azzurro. La variabile x_7 fa parte del backbone, in quanto raggiunge il suo complementare $\neg x_7$. Dopo l'eliminazione del backbone, il grafo rimanente è costituito da 2 WCC, rappresentate dai riquadri bianchi sullo sfondo.

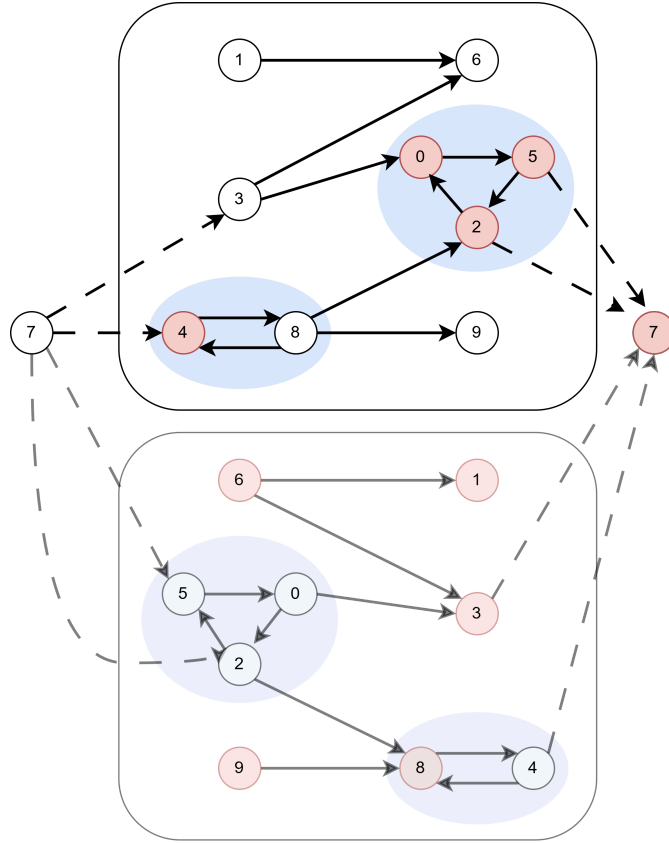


FIGURA 1. Esempio di grafo delle implicazioni

Dopo l'eliminazione dei backbone ci ritroviamo nella situazione descritta nella prima immagine di Figura 2, dove abbiamo evidenziato le WCC rimanenti. Quindi immaginiamo di applicare l'euristica 4, che prevede il ricalcolo dinamico della raggiungibilità dopo ogni assegnamento. Nella prima iterazione, i letterali $x_3, x_8, \neg x_6, x_5$ sono tutti dei candidati per essere aggiunti al fix-set, e quindi per essere eliminati, dato che

raggiungono tutti due nodi. Supponiamo di scegliere x_8 , dopo averlo assegnato a vero, e quindi eliminato, si propagano le assegnazioni di verità. Questo viene fatto anche nella WCC complementare, quindi $\neg x_8$ viene assegnato a falso e eliminato, e si propagano le assegnazioni di verità anche per i nodi che raggiungono $\neg x_8$. Dopo questa iterazione, la situazione è quella mostrata nella terza immagine di Figura 2, ora l'unico nodo candidato è $\neg x_6$, l'unico che raggiunge due nodi, quindi lo aggiungiamo al fix-set. Dopo averlo assegnato a vero e propagato le assegnazioni di verità abbiamo terminato in quanto non ci sono più nodi da assegnare. Il fix-set trovato è $X = \{x_8, x_6\}$, con assegnamenti $f(x_8) = \text{true}$, $f(x_6) = \text{false}$.

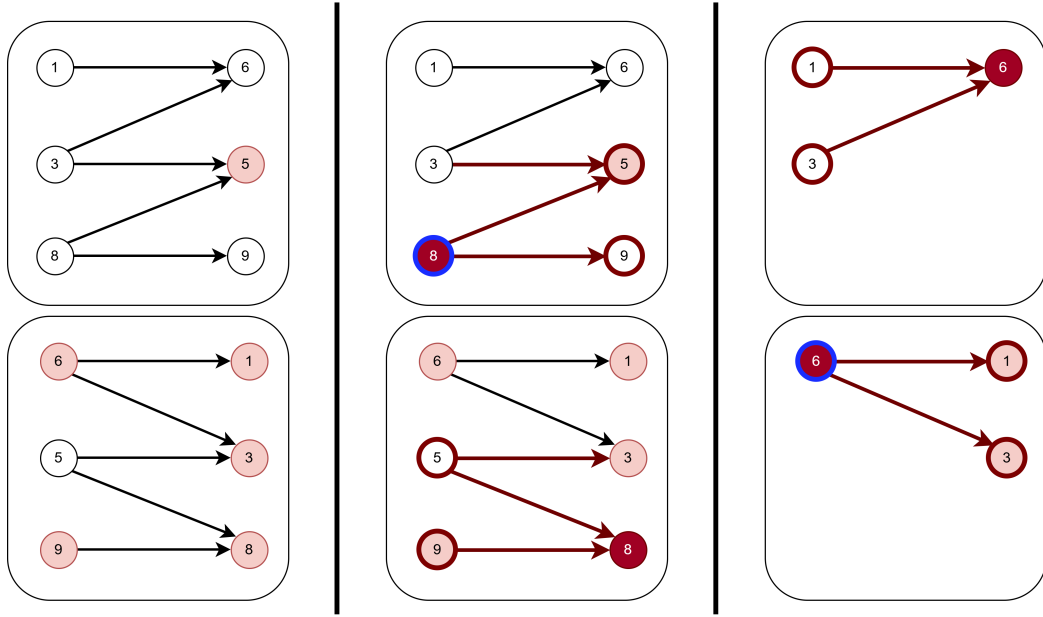


FIGURA 2. Applicazione dell'euristica 4 sull'esempio precedente

In caso di più coppie di WCC complementari possiamo scegliere un nodo per ogni coppia.

5. IMPLEMENTAZIONE

In questa sezione vengono descritte le scelte implementative più rilevanti, per maggiori dettagli si rimanda al codice sorgente.

5.1. Rappresentazione dei dati.

L'input viene letto da file in formato DIMACS CNF, da questo viene costruito il grafo delle implicazioni dove al letterale x corrisponde il nodo $2x$ e al letterale $\neg x$ corrisponde il nodo $2x + 1$. Il grafo viene rappresentato nel formato CSR (Compressed Sparse Row), che è una rappresentazione compatta ed efficiente per grafi sparsi, e permette di accedere rapidamente ai nodi adiacenti. Inoltre Il formato

CSR garantisce l'accesso coalescente alla memoria globale della GPU, poiché la memorizzazione contigua degli archi adiacenti permette ai thread CUDA di massimizzare il throughput in lettura. L'aspetto meno svantaggioso del formato CSR è che non permette di accedere rapidamente ai nodi che raggiungono un nodo, ma questo non sarà necessario nel nostro caso.

5.2. Calcolo delle SCC.

Per il calcolo delle SCC è stato implementato l'algoritmo proposto in [1], che viene riportato in Algoritmo 3. L'algoritmo assegna ad ogni nodo v due valori, v_{in} e v_{out} , alla fine della computazione i valori v_{in} e v_{out} saranno uguali e rappresenteranno l'identificatore della SCC a cui v appartiene.

L'algoritmo consiste in un ciclo esterno (Linee 2-21) che itera fino a quando la computazione è convergente. Ogni iterazione si articola in tre fasi: inizializzazione, propagazione dei valori massimi e rimozione degli archi.

La prima fase (Linee 3-6) inizializza i due valori di ciascun vertice all'ID del vertice corrispondente.

La seconda fase (Linee 7-14) propaga i valori massimi lungo gli archi. Per ogni arco diretto, il valore «out» del vertice sorgente viene aggiornato al valore «out» del vertice destinazione se quest'ultimo è maggiore. Analogamente, il valore «in» del vertice destinazione viene aggiornato al valore «in» del vertice sorgente se quest'ultimo è maggiore. Questa fase si ripete fino a raggiungere un punto fisso, ovvero finché nessun valore «in» o «out» cambia più.

La terza fase (Linee 15-19) rimuove gli archi i cui vertici sorgente e destinazione appartengono a SCC diverse. Rilevare questa condizione è semplice: se i valori del sorgente e della destinazione differiscono, i due vertici non appartengono alla stessa SCC e l'arco può essere rimosso. Le tre fasi si ripetono sul grafo ridotto, che ha gli stessi vertici ma meno archi.

L'algoritmo termina quando tutti i vertici hanno una firma in cui il valore «in» coincide con il valore «out».

Algoritmo 3: ECL-SCC

Input: Directed graph $G = (V, E)$ with unique vertex IDs
Output: $\forall v : v_{\text{in}}$ denotes to which SCC vertex v belongs

```
1  converged = false
2  while not converged do
3    for all vertices  $v \in V$  do
4       $v_{\text{in}} = v_{\text{id}}$ 
5       $v_{\text{out}} = v_{\text{id}}$ 
6    end for
7    updated = true
8    while updated do
9      for all edges  $(u, v) \in E$  do
10        $v_{\text{out}} = \max(v_{\text{out}}, u_{\text{out}})$ 
11        $v_{\text{in}} = \max(v_{\text{in}}, u_{\text{in}})$ 
12     end for
13     updated = at least one  $v_{\text{in}}$  or  $v_{\text{out}}$  value changed
14   end while
15   for all edges  $(u, v) \in E$  do
16     if  $u_{\text{in}} \neq v_{\text{in}}$  or  $u_{\text{out}} \neq v_{\text{out}}$  then
17        $E = E \setminus (u, v)$ 
18     end if
19   end for
20   converged = all  $v_{\text{in}} == v_{\text{out}}$ 
21 end while
```

Alla fine dell'algoritmo, l'identificatore della SCC corrisponde al nodo con ID più alto in quella SCC.

Ogni fase dipende dal risultato della fase precedente quindi non è possibile parallelizzare. La fase 1 è facilmente parallelizzabile e non richiede sincronizzazione, mentre la fase 3 richiede un **atomicAdd** per la costruzione della nuova lista di archi. La fase 2 è quella che richiede più tempo e può essere implementata con due operazioni atomiche: un **atomicMax** per aggiornare il valore «out» del vertice sorgente, e un **atomicMax** per aggiornare il valore «in» del vertice destinazione. Tuttavia gli autori hanno utilizzato un approccio che non utilizza operazioni atomiche, ma richiede più iterazioni per convergere, in questo modo l'algoritmo risulta più efficiente in pratica.

Dopo il calcolo delle SCC, è possibile costruire il grafo condensato, che è un DAG in cui ogni nodo rappresenta una SCC, e gli archi rappresentano le implicazioni tra le SCC. Per fare ciò gli ID delle componenti connesse vengono riassegnati nell'intervallo $[0, n)$, dove n è il numero di SCC, viene poi costruita una nuova lista di archi in cui ogni arco (u, v) del grafo originale viene trasformato in un arco $(scc_id[u], scc_id[v])$ se $scc_id[u] \neq scc_id[v]$. Da questa lista viene poi costruito il grafo condensato in formato CSR utilizzando alcune funzioni della libreria `thrust`.

La proprietà che l'ID di ogni SCC è dato dal nodo con ID più alto in quella SCC fa sì che le componenti siano ben ordinate in memoria. Infatti se la variabile con ID più alto in una SCC è u , e quindi l'SCC ha ID $2u$, allora l'SCC complementare che contiene $\neg u$ ha ID $2u + 1$, e lo stesso vale se la variabile con ID più alto in una SCC è $\neg u$, e quindi l'SCC ha ID $2u + 1$, allora l'SCC complementare che contiene u ha ID $2u$. Le SCC complementari si trovano quindi in posizioni adiacenti, come avveniva per i letterali, e sono sempre una pari e l'altra dispari.

Per trovare il complementare di una SCC con ID i sarà quindi sufficiente fare $i \text{ XOR } 1$, dove XOR è l'operazione XOR bit-a-bit.

5.3. Calcolo dell'ordinamento topologico.

Per l'ordinamento topologico è stata implementata una versione parallela dell'algoritmo di Kahn. La procedura si sviluppa iterativamente processando una «frontiera» di nodi con grado di ingresso nullo e rimuovendoli virtualmente dal grafo, aggiornando di conseguenza i gradi dei nodi adiacenti fino all'esaurimento dei vertici.

Durante l'attraversamento, l'algoritmo procede rigorosamente per livelli topologici. Ad ogni iterazione, i thread elaborano in parallelo la frontiera corrente visitando i nodi adiacenti e decrementandone attivamente i contatori delle dipendenze. Per garantire la correttezza sia il decremento dei gradi sia l'accodamento dei nuovi nodi che raggiungono un in-degree pari a zero vengono gestiti tramite le primitive atomiche.

5.4. Calcolo del backbone.

Per calcolare il backbone, come anticipato, è necessario verificare se un letterale può raggiungere il suo complementare nel DAG condensato. Per fare ciò, rispetto a quanto scritto nel Sezione 2, sono state implementate diverse accortezze.

L'allocazione diretta di una matrice di raggiungibilità di dimensione $O(n^2)$ risulta impraticabile su GPU per grafi di grandi dimensioni a causa dei limiti della VRAM. Invece di allocare un valore booleano per ogni singola query, lo stato di raggiungibilità viene compresso utilizzando interi a 64 bit (`unsigned long long`). Questo approccio permette di mappare e valutare simultaneamente fino a 64 query di raggiungibilità, corrispondenti ad altrettante coppie di nodi. Di conseguenza, l'operazione logica di disgiunzione descritta nello pseudocodice si traduce in una

singola istruzione hardware `atomicOr`, la quale incrementa drasticamente il throughput e riduce la memoria utilizzata.

Per prevenire errori di esaurimento della memoria su grafi di grandi dimensioni, le query vengono suddivise in batch da 64 elementi, i quali sono a loro volta raggruppati in chunk di dimensione calcolata dinamicamente per non eccedere un limite di VRAM prestabilito (ad esempio, 8 GB). L'host lancia iterativamente il kernel CUDA per ciascun chunk, garantendo la scalabilità dell'algoritmo indipendentemente dal numero totale di nodi. Grazie a questo design, un blocco di thread carica la lista di adiacenza di un nodo esattamente una volta per batch, ammortizzando il costoso prelievo in memoria globale su 64 esplorazioni simultanee e massimizzando così l'utilizzo della cache.

Infine, per garantire la correttezza della propagazione dei dati, l'elaborazione del grafo avviene rigorosamente livello per livello, basandosi sulle profondità estratte dall'ordinamento topologico. All'interno del kernel, i thread elaborano in parallelo i nodi appartenenti al medesimo livello e si sincronizzano esplicitamente (`__syncthreads()`) prima di procedere al livello successivo. Questa barriera previene l'insorgenza di race condition e assicura che, nel momento in cui un nodo propaga la propria maschera di raggiungibilità verso i figli, lo stato di tutti i suoi antenati sia già stato integralmente e definitivamente risolto in una singola passata.

Alla fine del processo viene ritornata una lista di assegnamenti `assignments`: per ogni nodo u , se u raggiunge il suo complementare, allora `assignments[u] = 0` (falso) e `assignments[complementare(u)] = 1` (vero), altrimenti `assignments[u] = -1` (non assegnato).

5.5. Calcolo delle WCC.

Per il calcolo delle Weakly Connected Components (WCC) sul grafo residuo, è stato implementato un classico algoritmo parallelo basato su pointer jumping noto in letteratura come approccio Hook-and-Compress [2]. L'algoritmo opera in modo iterativo associando inizialmente ogni nodo a se stesso come radice. Successivamente, per ogni arco del grafo, la componente con l'identificativo maggiore viene annessa («hooking») a quella con l'identificativo minore tramite operazioni atomiche, seguita da una fase di compressione dei cammini («compress») per appiattire gli alberi risultanti. Il kernel esclude attivamente dall'elaborazione tutti i nodi nel backbone.

Una volta raggiunta la convergenza e appiattiti gli alberi, è necessario raggruppare i nodi appartenenti alla medesima componente per facilitare le elaborazioni successive. A tale scopo, è stata impiegata la libreria Thrust per ordinare i nodi in base all'ID della propria WCC e calcolarne le dimensioni tramite una riduzione (`reduce_by_key`). Il risultato finale viene infine riorganizzato e restituito utilizzando una struttura dati in formato CSR, dove l'array `row_ptr` delimita l'inizio e la fine di ciascuna componente, mentre `col_ind` contiene gli identificativi dei nodi raggruppati, garantendo così accessi di memoria coalescenti nelle fasi successive della pipeline.

5.6. Calcolo delle euristiche.

Il calcolo delle metriche per le euristiche 1 e 2 è relativamente semplice, in quanto si tratta di contare il numero di archi uscenti (out-degree) o di identificare i nodi sorgente, operazioni che possono essere eseguite in parallelo senza dipendenze tra i thread.

Per le euristiche basate sulla raggiungibilità (Heuristic 3 e 4), il conteggio per i nodi raggiungibili avviene propagando l'informazione nel ordine topologico inverso (dalle foglie verso le radici), dove i vertici aggregano parallelamente le maschere di bit dei propri figli, in modo simile a quanto è stato fatto per il calcolo dei backbone con la differenza che in questo caso non viene controllata la raggiungibilità verso il complementare, ma semplicemente si conta il numero di nodi raggiungibili, cioè i bit attivi nella maschera di raggiungibilità. Al fine di rispettare i limiti della VRAM, anche questa fase è strutturata a blocchi come per il caso del calcolo dei backbone.

Una volta calcolati i punteggi, la selezione del candidato ottimale all'interno di ogni singola WCC è delegata alla libreria Thrust: tramite una riduzione parallela (`reduce_by_key`), l'algoritmo raggruppa i nodi per ID della componente ed estrae simultaneamente il vertice con il valore massimo. Successivamente, un altro kernel analizza le coppie di WCC complementari, decreta il vertice vincitore assoluto per ciascuna coppia e lo inserisce in una coda. Da qui prende avvio la fase di propagazione delle assegnazioni per ogni WCC, che segue il pattern della visita in ampiezza. I thread prelevano i nodi dalla coda e ne ispezionano i vicini: se un vertice adiacente risulta non ancora assegnato, il thread ne forza l'assegnamento (e lo stesso per il suo complementare) in modo sicuro e concorrente, avvalendosi della primitiva `atomicCAS` (Compare-And-Swap). I nodi modificati con successo vengono quindi accodati per la frontiera successiva, e il processo continua iterativamente fino a quando tutte le assegnazioni sono state propagate e non rimangono più nodi da processare.

6. RISULTATI SPERIMENTALI

Per valutare le prestazioni dell'implementazione, sono state condotte una serie di analisi su istanze di 2-SAT. Viene utilizzata una macchina virtuale, con sistema operativo Debian 11, con le seguenti caratteristiche hardware:

Device Name	Nvidia Tesla T4	Intel Xeon E5 v3
Mirco-Architecture	Turing	Haswell
SMs / Sockets	40	1
Cores	2560 CUDA cores	2 cores with 2 threads each
Boost Frequency	1.59 GHz	3.6 GHz
Peak FLOPS (FP32)	8.1 TFLOPS	147 GFLOPS
Memory	16 GB GDDR6	15GB DDR4
Cache	6 MiB (L2 Cache)	45 MiB (L3 Cache)

6.1. Metodo di generazione delle istanze.

Per generare un istanza di 2-SAT con n variabili e m clausole, viene prima generato un assegnamento per le n variabili. Per ogni clausola da generare vengono scelti due letterali a caso, e viene controllato che almeno uno dei due letterali sia vero nell'assegnamento generato, in questo modo si garantisce che l'istanza sia soddisfacibile. Per ogni istanza è possibile quindi specificare sia il numero di variabili che il numero di clausole, si è quindi controllato come il grafo delle implicazioni cambia al variare del rapporto tra clausole e variabili, e come questo impatta sulle prestazioni dell'algoritmo. Al variare del rapporto e del numero di variabili, sono stati contati il numero di SCC e il numero di WCC. Si è subito notato che ogni conteggio aumenta linearmente con l'aumentare del numero di variabili, invece il rapporto clausole/variabili ha un impatto significativo.

Per visualizzare meglio questi risultati, è stata generata una griglia di istanze con numero di variabili che va da 100 a 5000, e rapporto clausole/variabili che va da 0.5 a 4.5. Le istanze variano 41 volte per quanto riguarda il numero di variabili, e 41 volte per quanto riguarda il rapporto clausole/variabili, per un totale di 1681 istanze. Per ogni istanza sono stati contati il numero di SCC e il numero di WCC dopo l'eliminazione dei backbone.

Nella Figura 3 viene mostrato come il rapporto tra il numero di SCC e il numero di letterali varia rispetto al rapporto clausole/variabili. Per un rapporto $r < 2$ (e quindi un rapporti archi/nodi nel grafo delle implicazioni < 1) il numero di SCC è uguale al numero di letterali, mentre per $r > 2$ inizia a diminuire.

Per quanto riguarda il numero di WCC per SCC, mostrato nella Figura 4, si nota che per $r < 2$ la funzione è decrescente, l'aggiunta di archi nel grafo delle implicazioni aggrega sempre più nodi e quindi sempre più componenti debolmente connesse. Per $r > 2$ invece la funzione è crescente, questo per l'effetto dell'eliminazione dei backbone. Infatti, senza la loro eliminazione, il numero di WCC diminuirebbe fino a 1, dato che il grafo delle implicazioni diventerebbe sempre più connesso. L'eliminazione dei backbone diventa sempre più frequente all'aumentare del numero degli archi e quindi frammenta sempre di più il grafo, aumentando il numero di WCC.

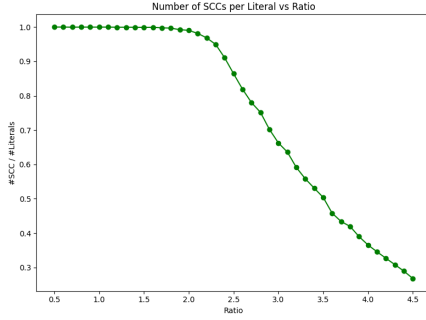


FIGURA 3. Il rapporto tra il numero di SCC e il numero di letterali al variare del rapporto clausole/variabili (Ratio)

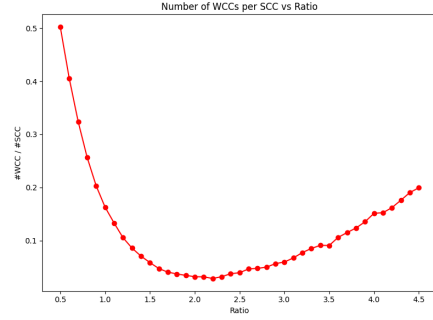


FIGURA 4. Il rapporto tra il numero di WCC e il numero di SCC al variare del rapporto clausole/variabili (Ratio)

6.2. Grado di approssimazione delle euristiche.

Per valutare il grado di approssimazione delle euristiche, è stato confrontato il fix-set trovato da ogni euristica con un fix-set ottimale ottenuto tramite risoluzione esatta con Clingo, un solver ASP (Answer Set Programming) che permettere di scrivere i vincoli del problema in modo semplice e di ottenere soluzioni ottimali.

Sono state generate 41 istanze con rapporto clausole/variabili che va da 0.5 a 4.5 con 500 variabili per 10 volte. Per ogni istanza è stato calcolato il fix-set ottimale con Clingo, con un timeout di 400 secondi, e il fix-set approssimato con ogni euristica. Il tempo di esecuzione di Clingo sulle varie istanze può essere visualizzato nella Figura 5, per rapporti tra 1.1 e 2.7 il tempo impiegato è notevolmente più alto.

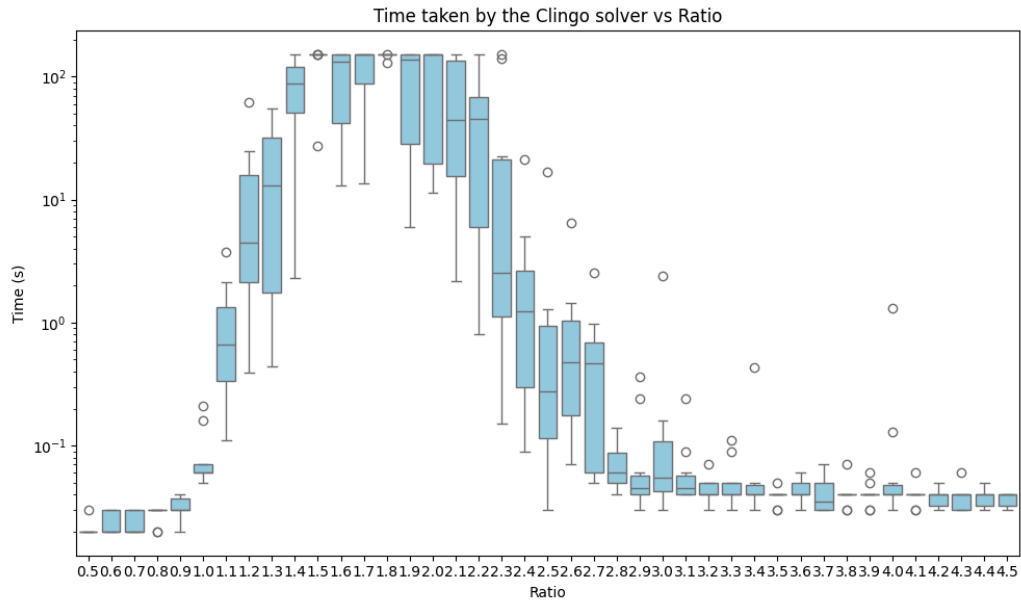


FIGURA 5. Tempo di esecuzione di Clingo al variare del rapporto clausole/variabili, tempo in scala logaritmica

Per ogni istanza è stato calcolato il rapporto tra la dimensione del fix-set approssimato e la dimensione del fix-set ottimale, e sono stati calcolati i valori medi di questo rapporto per ogni euristica al variare del rapporto clausole/variabili. I risultati sono mostrati nella Figura 6.

L'euristica 1, che seleziona un nodo sorgente, è quella che si discosta maggiormente dall'ottimo. L'euristica 2 e 3 hanno un comportamento simile nelle regioni dove il problema è più difficile, mentre l'euristica 3 sembra avere il vantaggio altrimenti. L'euristica 4, che prevede il ricalcolo dinamico della raggiungibilità, è quella che si avvicina maggiormente all'ottimo, con un rapporto medio che si mantiene sempre inferiore a 1.25, e che si avvicina a 1 per rapporti tra clausole e variabili più alti.

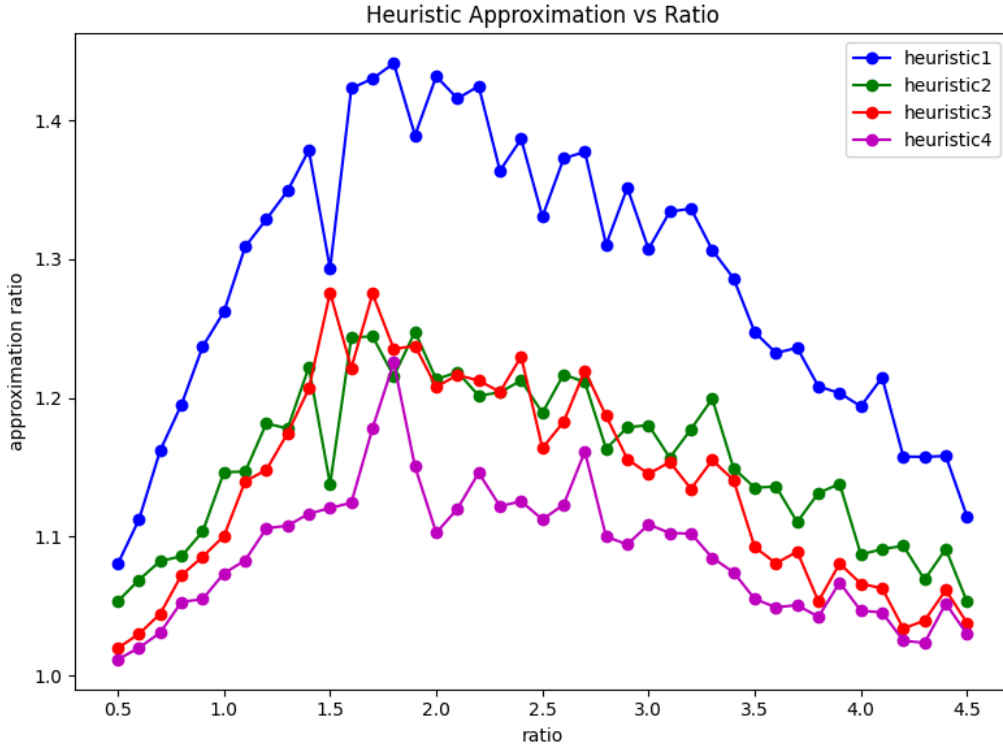


FIGURA 6. Il rapporto tra la dimensione del fix-set approssimato e la dimensione del fix-set ottimale al variare del rapporto clausole/variabili, per ogni euristica

6.3. Analisi delle performance.

Per valutare le prestazioni dell'implementazione, sono stati condotti benchmark sulle stesse istanze utilizzate nella Sezione 6.1.

Nella Figura 7 viene mostrato il tempo di esecuzione totale delle prime fasi dell'algoritmo, ovvero il calcolo delle SCC, del backbone e delle WCC, al variare del rapporto clausole/variabili e dal numero di variabili. Si può notare come il tempo di esecuzione aumenti con il numero di variabili, ed inoltre il tempo impiegato per

istanze con rapporto clausole/variabili > 2 è superiore a quello impiegato per istanze con rapporto clausole/variabili < 2 . Inoltre il tempo di esecuzione di queste prime fasi è nettamente inferiore al tempo utilizzato dalla fase di assegnamento guidato dalle euristiche, che è quella più costosa.

Nella figura 8 invece riportato il tempo impiegato dall'euristica 4, per le altre euristiche l'heatmap è simile, ma con tempi di esecuzione inferiori. Anche in questo caso si nota come il tempo di esecuzione aumenti con l'aumentare del numero di variabili. Invece al variare del rapporto il tempo di esecuzione si comporta in modo simile a quello mostrato nella Figura 5, il picco del tempo di esecuzione si ha intorno al rapporto di 1.2 e si riduce più velocemente per rapporti più piccoli rispetto a rapporti più grandi.

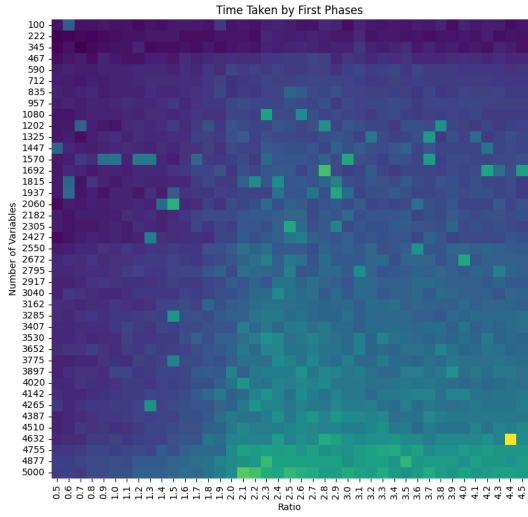


FIGURA 7. Heatmap del tempo di esecuzione totale delle prime fasi dell'algoritmo, al variare del rapporto clausole/variabili e dal numero di variabili

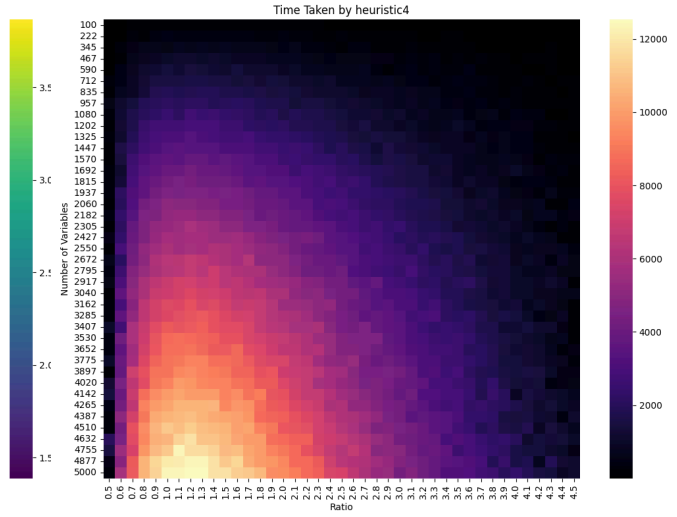


FIGURA 8. Heatmap del tempo di esecuzione dell'euristica 4, al variare del rapporto clausole/variabili e dal numero di variabili

Per confrontare il tempo di esecuzione delle 4 euristiche sono state generate 200 istanze con un numero di variabili che va da 1000 a 25000, e con un rapporto clausole/variabili di 1.8. I risultati sono mostrati nella Figura 9, l'euristica 4 è quella che impiega più tempo in assoluto, quasi 100 volte più delle altre euristiche, il calcolo delle raggiungibilità ad ogni iterazione risulta quindi molto costoso anche se permette di avvicinarsi maggiormente all'ottimo.

La scala utilizzata è doppiamente logaritmica per poter confrontare al meglio i tempi di esecuzione, per fare in modo che i punti siano equidistanti in questa visualizzazione il numero di variabili delle istanze è stato generato con una distribuzione log-uniforme.

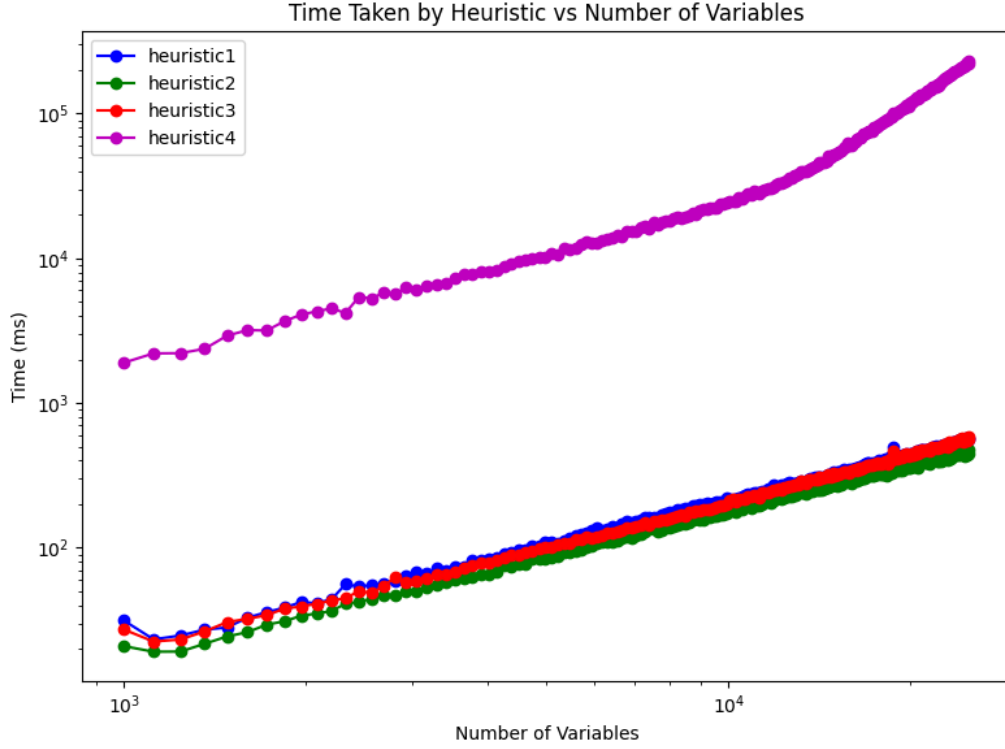


FIGURA 9. Confronto del tempo di esecuzione delle 4 euristiche al variare del numero di variabili, scala doppiamente logaritmica

Sono stati quindi condotti benchmark su istanze più grandi per le prime tre euristiche, con un numero di variabili che va da 10^3 a 10^6 , e con un rapporto clausole/variabili di 1.8.

Nella Figura 10 viene mostrato il confronto del tempo di esecuzione delle 3 euristiche più veloci, al variare del numero di variabili, con una scala lineare a sinistra e una scala doppiamente logaritmica a destra. Si nota come verso le 10^5 variabili la pendenza delle rette nel grafico con scala doppiamente logaritmica aumenti. Il punto corrisponde a quando il limite di memoria scelto viene superato e quindi è necessario suddividere le query di raggiungibilità in più chunk, questo comporta un aumento del tempo di esecuzione, ma permette di scalare a istanze più grandi. Inoltre è interessante calcolare la pendenza delle rette prima e dopo questo punto, infatti se il tempo di esecuzione è un polinomio del tipo $t = c * n^k$ allora la pendenza della retta in scala logaritmica è k :

$$\log(t) = \log(c * n^k) = \log(c) + k * \log(n)$$

Nella Tabella 2 vengono riportati i valori stimati per la pendenza delle rette prima e dopo il punto in cui il numero di chunk è maggiore di 1.

Prima di questo punto il regime è molto vicino a quello lineare, mentre dopo questo punto la pendenza aumenta, ma rimane comunque inferiore a 2, quindi il

tempo di esecuzione sembra scalare in modo sub-quadratico anche per istanze molto grandi.

Dai test effettuati l'euristica 2 risulta essere la più veloce, subito seguita dall'euristica 3 e poi dall'euristica 1.

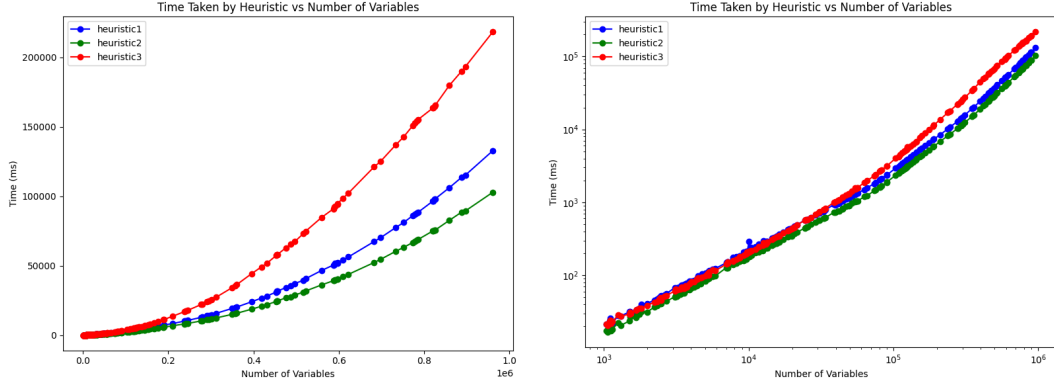


FIGURA 10. Confronto del tempo di esecuzione delle 3 euristiche più veloci al variare del numero di variabili. Sulla destra scala doppiamente logaritmica

	1 chunk	>1 chunk
Heuristic 1	1.063	1.745
Heuristic 2	1.060	1.723
Heuristic 3	1.137	1.837

TABELLA 2. Valori stimati per la pendenza delle rette prima e dopo il punto in cui il numero di chunk è maggiore di 1

7. POSSIBILI MIGLIORAMENTI

Il metodo con cui sono state generate le istanze tende a creare una grande componente debolmente connessa, quindi dopo poche iterazioni di assegnazioni delle variabili rimane un'unica WCC e la parallelizzazione viene a meno. Per ovviare a questo problema, si potrebbe pensare ad un metodo per assegnare più variabili in ogni iterazione in un singolo WCC e quindi gestire in modo corretto gli eventuali conflitti. Se invece l'obiettivo è quello di raggiungere una migliore approssimazione a discapito del tempo di esecuzione una beam search potrebbe essere una buona soluzione, la ricerca può essere eseguita in parallelo e guidata dalle stesse euristiche.

8. VALUTAZIONI E OSSERVAZIONI

L'euristica 4, che prevede il ricalcolo dinamico della raggiungibilità, è quella che si avvicina maggiormente all'ottimo, tuttavia questa euristica è anche quella che

impiega più tempo in assoluto, quasi 100 volte più delle altre euristiche, il calcolo delle raggiungibilità ad ogni iterazione risulta quindi molto costoso. L'euristica 2 e 3 hanno un comportamento simile per quanto riguarda il grado di approssimazione ma l'euristica 2 è significativamente più veloce dell'euristica 3, quindi sembra essere la scelta migliore se si vuole un buon compromesso tra qualità della soluzione e tempo di esecuzione. L'euristica 1, che seleziona un nodo sorgente, è quella che si discosta maggiormente dall'ottimo e impiega più tempo rispetto all'euristica 2, quindi sembra essere la scelta peggiore tra le 4 euristiche.

9. GUIDA ALL'USO

Per compilare è sufficiente eseguire `make`, dopo la compilazione, si otterrà un eseguibile chiamato `fix`, che può essere utilizzato per risolvere istanze di 2-SAT e trovare un fix-set approssimato.

Esecuzione:

```
./fix <percorso_istanza.cnf> [heuristic list] [--check-sodd] [--bench] [--bench-file <path>]
```

- `<percorso_istanza.cnf>`: Il percorso al file contenente l'istanza di 2-SAT in formato CNF.
- `[heuristic list]`: Una lista opzionale di euristiche da applicare, ad esempio `--heuristics 1,3` per applicare le euristiche 1 e 3. Se non viene specificata alcuna euristica, verranno applicate tutte le euristiche disponibili.
- `--check-sodd`: Opzione per verificare se l'istanza è soddisfacibile
- `--bench`: Opzione per abilitare il benchmarking delle prestazioni.
- `--bench-file <path>`: Specifica un file di output per i risultati del benchmark, se non specificato i risultati del benchmark verranno salvati su `benchmarks.csv`.

Sono stati utilizzati anche due script python per generare istanze di 2-SAT e per eseguire i benchmark su più istanze in modo automatico. Questi script si trovano nella cartella `scripts/`.

Per la generazione di istanze soddisfacibili:

```
python scripts/generate_instances.py <num_var_start> <num_var_end> <num_var> <ratio_start> <ratio_end> <num_ratio> <output_dir> [--clingo-timeout <seconds>]
```

- `<num_var_start>`: Numero minimo di variabili per le istanze generate.
- `<num_var_end>`: Numero massimo di variabili per le istanze generate.
- `<num_var>`: Numero di istanze, rispetto al numero di variabili, da generare.
- `<ratio_start>`: Rapporto minimo tra clausole e variabili per le istanze generate.
- `<ratio_end>`: Rapporto massimo tra clausole e variabili per le istanze generate.

- `<num_ratio>`: Numero di istanze, rispetto al rapporto clausole/variabili, da generare.
- `<output_dir>`: Directory di output dove salvare le istanze generate.
- `--clingo-timeout <seconds>`: Timeout in secondi per la risoluzione dell'istanza tramite Clingo, se non specificato, clingo non viene eseguito.

All'interno del file inoltre si possono cambiare SEED ed il metodo per generare il numero delle istanze.

Per eseguire i benchmark su più istanze:

```
python scripts/run_benchmark.py <instances dir> <heuristic list> <out file>
[--check-sodd]
```

- `<instances dir>`: Directory contenente le istanze da testare.
- `<heuristic list>`: Una lista di euristiche da applicare, ad esempio `--heuristics 1,3` per applicare le euristiche 1 e 3.
- `<out file>`: File di output per i risultati del benchmark, ad esempio `benchmark_results.csv`.
- `--check-sodd`: Opzione per verificare se le istanze sono soddisfacibili prima di eseguire il benchmark.

9.1. Riproducibilità.

Per rieseguire i benchmark completi si possono utilizzare i comandi commentati nella funzione «main» degli script di generazione dei dati e di benchmarking.

REFERENCES

1. Alabandi, G., Sands, W., Biros, G., Burtcher, M.: A GPU Algorithm for Detecting Strongly Connected Components, <https://doi.org/10.1145/3581784.3607071>
2. Soman, J., Kishore, K., Narayanan, P. J.: A fast GPU algorithm for graph connectivity. In: 2010 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum (IPDPSW). pp. 1–8 (2010)